

Instructions for the mini-lab on “basic models”.

1. Familiarize yourself with the basics of Typst, as the report must be written in Typst. See at the end of this document for a quick meta-intro.
2. Get the provided python code, run it, at least in “HEADLESS” mode (so that it does not need any GUI interaction and just generate figures as png files).
3. In your report explain what you understand from the behavior of different models:
 - what is each model doing (the names might not be full explanatory),
 - what behavior do you observe on the different datasets, e.g. in 2D with varied complexity but also on MNIST derivatives,
 - what limitations/constraints do you observe in the models? How do you see that in the results?
 - what further experiments did you do?
 - etc...
4. Your report should be clear and synthetic
 - you can reference image file names if you kept the code untouched,
 - you don't need to comment each figure, but rather summarize the behavior you observe,
5. You will submit a zip file with
 - your typst report file (e.g., `report-lab1-yourlastname.typ`) source file,
 - the pdf file (e.g., `report-lab1-yourlastname.pdf`) generated,
 - any code that you might have written/modified, and the corresponding figures (if any).

Here is an example of what you could write:

“

- ...
- A spherical Gaussian is constrained to a single variance parameter. As shown in Fig. 16, it thus cannot fix an anisotropic Gaussian distribution. It generates samples that are too spread in one direction and too concentrated in the other.
- In the case of MNIST0, the spherical gaussian behavior (corresponding to Fig. 116) now appears in Fig. 47 as ...
- ...

”

Basics of typst

- Typst is like latex but rebuilt from scratch to be simpler and more modern.

- You can use it online at <https://typst.app/> or install it locally from <https://typst.org/> and use e.g. the vscode “tinymist” plugin.
- It uses a markdown-like syntax (but-not-exactly) for very common things, like paragraphs, lists, and headers.
- It has modes
 - by default, it is in “text mode”, where you can write text, lists, headers, etc.
 - example **bold** and *italic* text and `monospace` text.
 - you can enter code mode using `#` and use functions, variables, etc.
 - example computation `#(1 + 1)` gives `2`.
 - example function call `#text(orange, "hello")` gives `hello`.
 - you can switch back to text mode within code mode using square brackets `[]`, e.g., `#text(blue, [*1*+_1_])` yields `1+1`.
 - you can enter “math mode” by using `$. . . $` for inline math,
 - the syntax is different from latex (for good reasons)
 - to convert latex to typst, use e.g. <https://qwins.github.io/tex2typst-webapp/>.
 - inline math `$E=m c^2$` gives $E = mc^2$.
 - block math `$ E=m c^2 $` with spaces after/before dollars

$$E = mc^2$$

- fractions and groups made simpler
 `$4 / 3 pi R^3 = (4 pi R^3) / 3$` yields $\frac{4}{3}\pi R^3 = \frac{4\pi R^3}{3}$.
- in math mode, one can use variables (like any of the greek letters) or symbols, potentially with modifiers... e.g.
 `$ EE_(x tilde q) [f(x)] eq.def integral_x f(x) q(x) d x $`

$$\mathbb{E}_{x \sim q}[f(x)] \stackrel{\text{def}}{=} \int_x f(x)q(x)dx$$
- you can use quotes to insert text in math mode, e.g.,
 `$E = "mass" times c^2$` gives $E = \text{mass} \times c^2$
- you can also call functions, the parameters are passed “raw”, so
 `$sqrt(x)$` yields $\sqrt{x}$ as it does not consider `x` as a variable but rather as raw input
- you can switch back to code mode if needed, e.g.,
 `$sqrt( #(str(1 + 1)) )$` gives $\sqrt{2}$ (needing an explicit cast).
- And much more including custom functions, show rules, set rules, diagrams/ plots with `cetz`, loading data from `yaml` files...